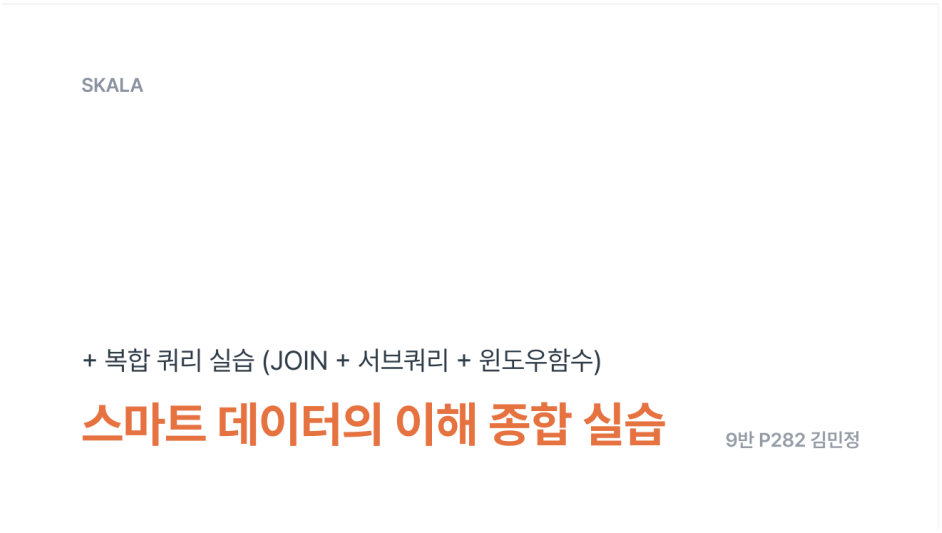
복합 쿼리 실습 (JOIN + 서브쿼리 + 윈도우함수)

날짜	2026년 7월 28일
태그	



목차

목차
배경
제출물
1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회
코드
결과
2.모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기
코드
결과
3. 수강이 기준. 학생이 없으면 학생 정보가 NULL - outer join
코드
결과
4. 학생/수강 모두 포함
코드
결과
5. 한 번도 수강하지 않은 학생 목록
코드
결과
6. 한 과목 이상 수강한 학생 목록(중복 제거) - 중복 제거만
코드
결과
7. 고객별 주문건수/총액
코드
결과

8. 총액 상위 10명과 금액

코드

결과

9. 모든 직원과 그 매니저 이름

코드

결과

10. "모든 학생 기준"으로 과목 분포를 보고 싶다 → LEFT JOIN + 집계

코드

결과

11. DB 과목을 듣지 않은 모든 학생을 나열

코드

결과

12. (가정) 과목별로 매니저가 운영 책임을 갖는다고 가정하고,emp의 매니저(이름 'Mgr_'로 시작)와 과목을 임의로 매핑한 테이블 course_owner(course, manager_id)를 만든 뒤,과목별 수강 인원 + 책임 매니저 이름 리포트를 작성하세요

코드

결과

13. 학생 × 과목 전체 조합을 만들어 "학생별 과목 추천 후보"를 만들되, 샘플 100건만 본다

코드

결과

14. 스칼라 서브쿼리 (SELECT 절) 사용 학생 + 소속 학과명 붙이기

코드

결과

15. 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)

코드

결과

16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)

코드

결과

17. 수강(enroll) 기록이 있는 학생만

코드

결과

18. 한 번도 수강하지 않은 학생

코드

결과

19. HR 학과 학생 일부와의 비교 데모

코드

결과

20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록

코드

결과

21. student 테이블에는 학과(major)와 GPA가 있습니다. 학과별·GPA 구간별 인원을 집계하되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력

코드

결과

22. emp 테이블은 CEO(manager_id = NULL) → 매니저(10명) → 개발자(300명)의 3단계 계층 구조입니다. WITH RECURSIVE로 모든 직원의 계층 경로와 깊이를 출력 - 그룹없을 때 널값이 나오지 않도록 전체 소계

코드

결과

코드

결과

23. 각 학과(major)별로 GPA가 높은 순서로 순위를 매기고, 상위 3명씩만 추출하는 쿼리를 Window Function으로 작성 (서브쿼리 방식과 CTE 방식 모두 작성)

코드

결과

24. enroll 테이블을 student_id 기준으로 정렬하고, 각 학생의 이전 수강 과목 대비 성적 변화를 LAG()로 계산 (성적은 A=4, B=3, C=2, D=1 점수로 변환)

코드

결과

25. orders 테이블의 주문을 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평 균 계산 (ROWS BETWEEN 사용)

코드

결과

코드

결과

배경

- 학사 시스템: **student, enroll**
  - 규칙상 수강은 학생당 0~2건
  - student_id % 3 = 0 → 0건 (약 333명 수강 없음)
  - = 1 → 1건 (약 334명)
  - = 2 → 2건 (약 333명)
  - 또한 고아 수강(enroll에만 존재)
  2건(학생 1001, 1010) 포함
- 캠퍼스 스토어: **customers, orders**
  - 고객 1명당 정확히 6건의 주문이 존재
- 조직도: **emp**
  - CEO 1명, 매니저 10명, 직원 300명
  (각 직원은 매니저 10명 중 1명에게 배정)

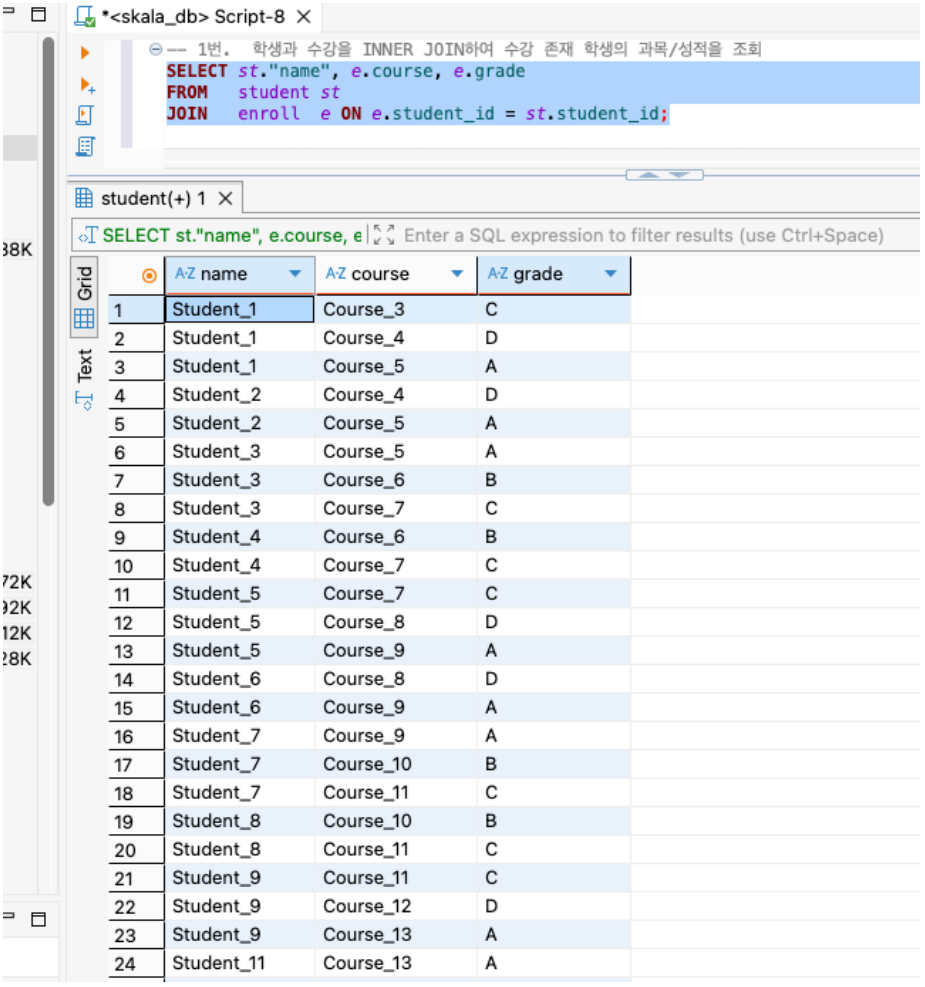
제출물

1. 학생과 수강을 INNER JOIN하여 수강 존재 학생의 과목/성적을 조회

코드

```
SELECT st."name", e.course, e.grade
FROM   student st
JOIN   enroll  e ON e.student_id = st.student_id;
```

결과

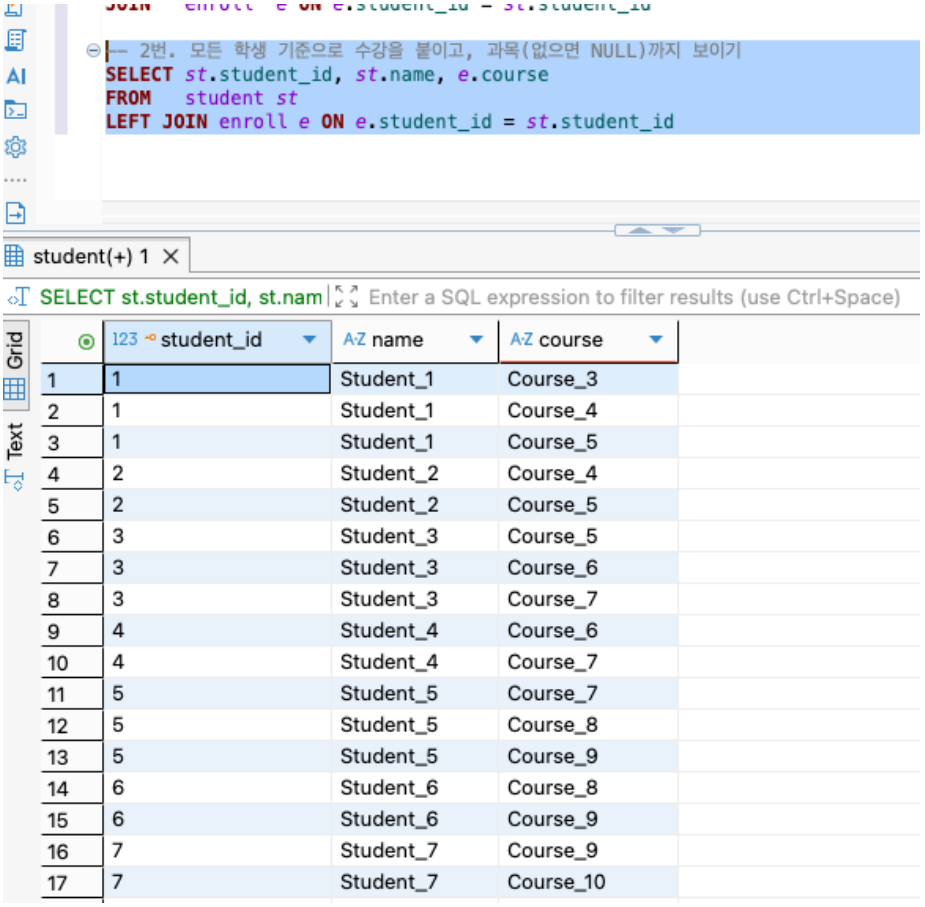


2.모든 학생 기준으로 수강을 붙이고, 과목(없으면 NULL)까지 보이기

코드

```
SELECT st.student_id, st.name, e.course
FROM   student st
LEFT JOIN enroll e ON e.student_id = st.student_id
```

결과



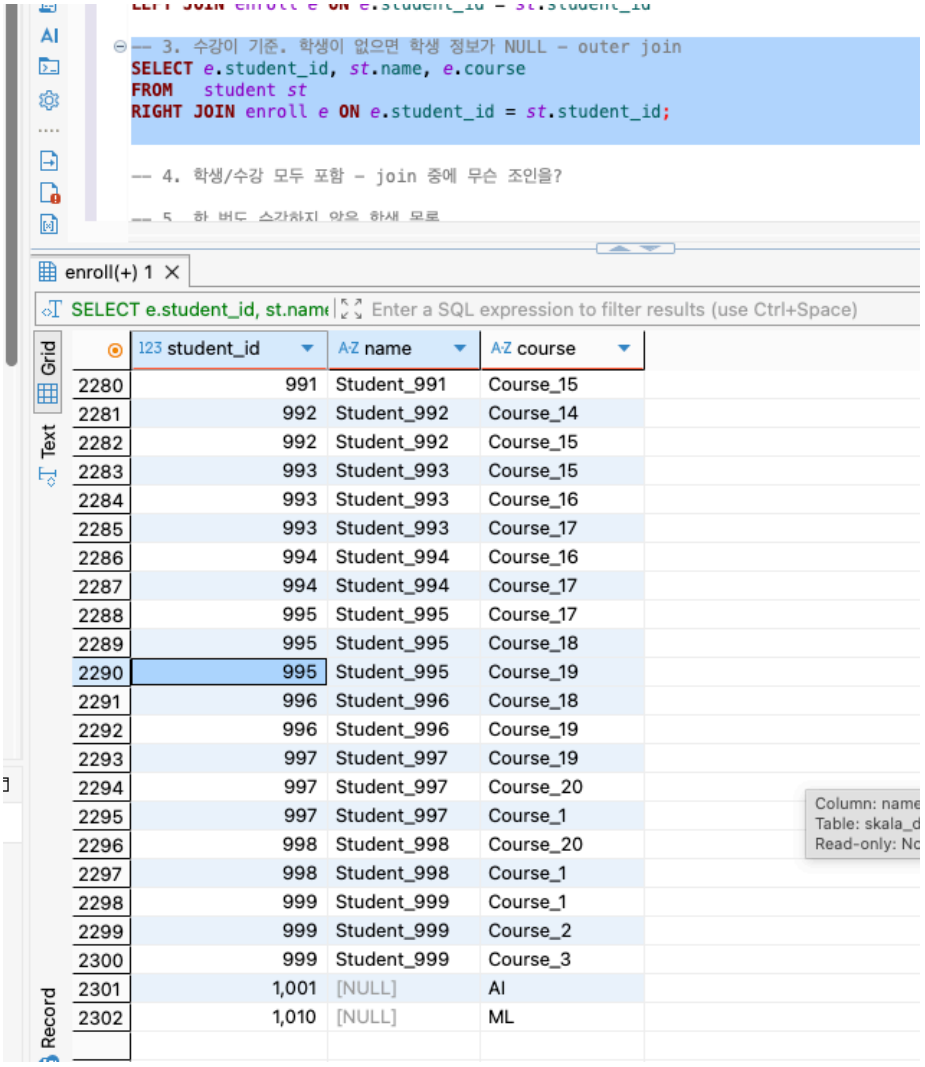
	student_id	name	course
1	1	Student_1	Course_3
2	1	Student_1	Course_4
3	1	Student_1	Course_5
4	2	Student_2	Course_4
5	2	Student_2	Course_5
6	3	Student_3	Course_5
7	3	Student_3	Course_6
8	3	Student_3	Course_7
9	4	Student_4	Course_6
10	4	Student_4	Course_7
11	5	Student_5	Course_7
12	5	Student_5	Course_8
13	5	Student_5	Course_9
14	6	Student_6	Course_8
15	6	Student_6	Course_9
16	7	Student_7	Course_9
17	7	Student_7	Course_10

3. 수강이 기준. 학생이 없으면 학생 정보가 NULL - outer join

코드

```sql
SELECT e.student_id, st.name, e.course
FROM   student st
RIGHT JOIN enroll e ON e.student_id = st.student_id;
```

결과

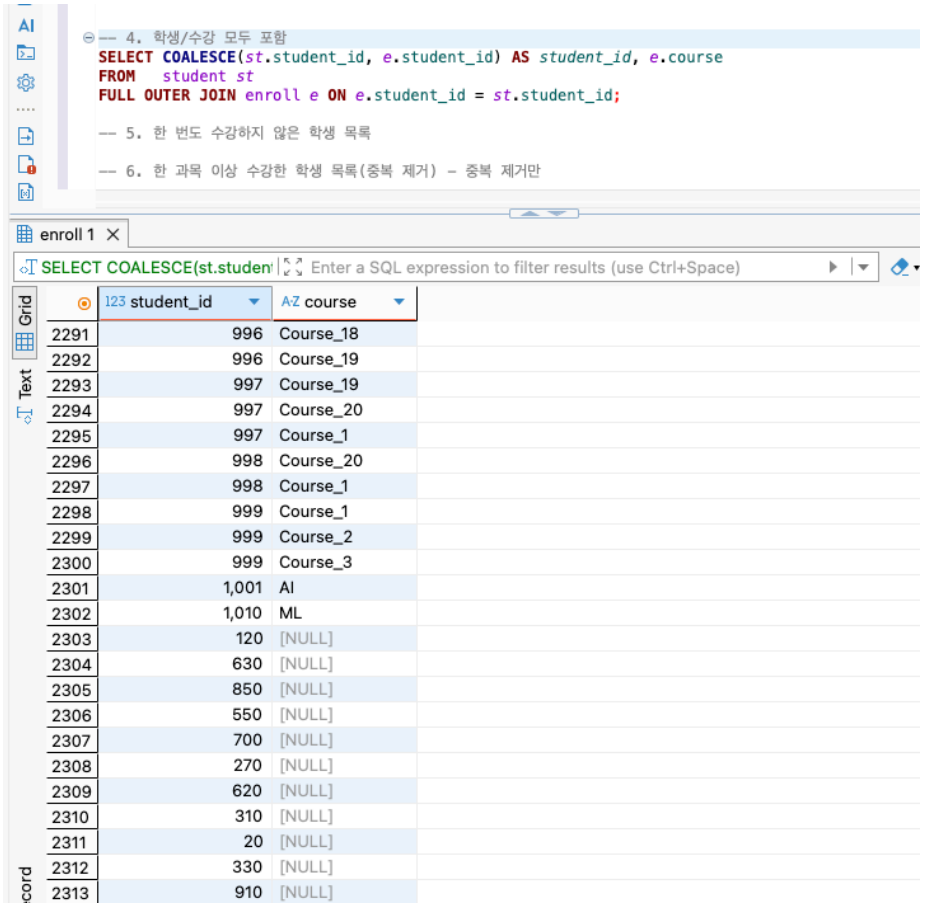


4. 학생/수강 모두 포함

코드

```sql
SELECT COALESCE(st.student_id, e.student_id) AS student_id, e.course
FROM   student st
FULL OUTER JOIN enroll e ON e.student_id = st.student_id;
```

결과



```
-- 4. 학생/수강 모두 포함
SELECT COALESCE(st.student_id, e.student_id) AS student_id, e.course
FROM   student st
FULL OUTER JOIN enroll e ON e.student_id = st.student_id;

-- 5. 한 번도 수강하지 않은 학생 목록

-- 6. 한 과목 이상 수강한 학생 목록(중복 제거) - 중복 제거만
```

enroll 1

SELECT COALESCE(st.studen | Enter a SQL expression to filter results (use Ctrl+Space)

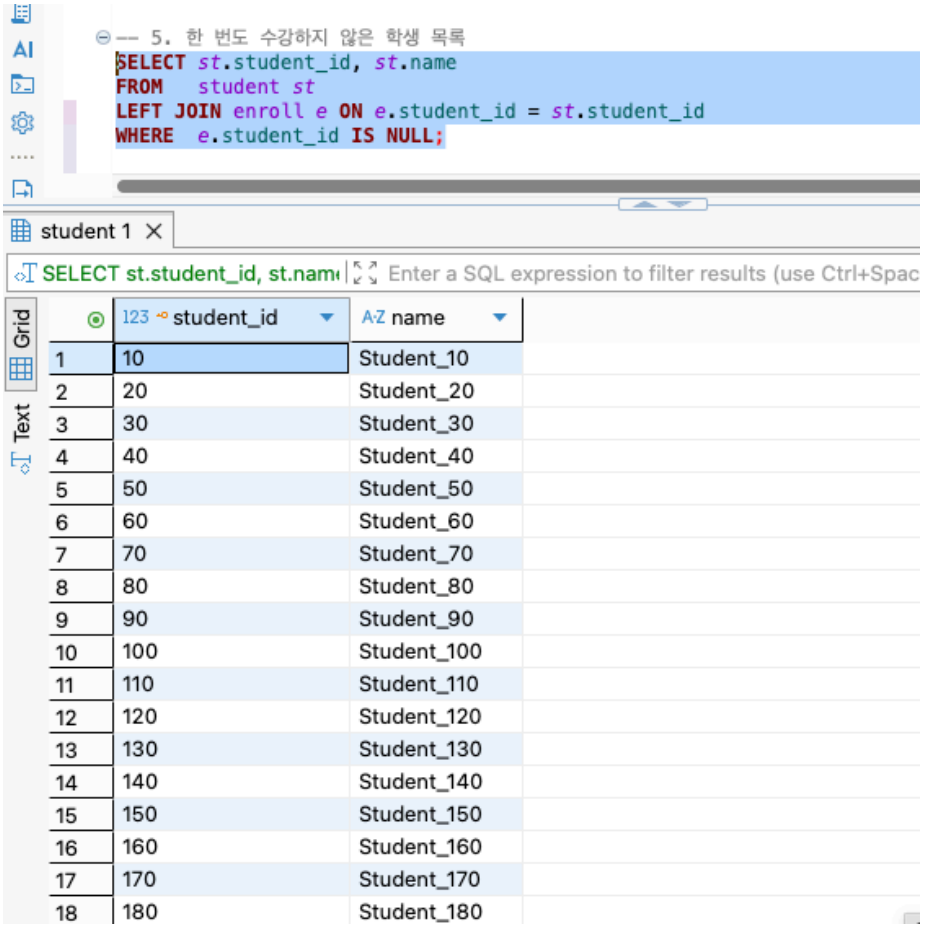
	student_id	course
2291	996	Course_18
2292	996	Course_19
2293	997	Course_19
2294	997	Course_20
2295	997	Course_1
2296	998	Course_20
2297	998	Course_1
2298	999	Course_1
2299	999	Course_2
2300	999	Course_3
2301	1,001	AI
2302	1,010	ML
2303	120	[NULL]
2304	630	[NULL]
2305	850	[NULL]
2306	550	[NULL]
2307	700	[NULL]
2308	270	[NULL]
2309	620	[NULL]
2310	310	[NULL]
2311	20	[NULL]
2312	330	[NULL]
2313	910	[NULL]

5. 한 번도 수강하지 않은 학생 목록

코드

```sql
SELECT st.student_id, st.name
FROM   student st
LEFT JOIN enroll e ON e.student_id = st.student_id
WHERE  e.student_id IS NULL;
```

결과



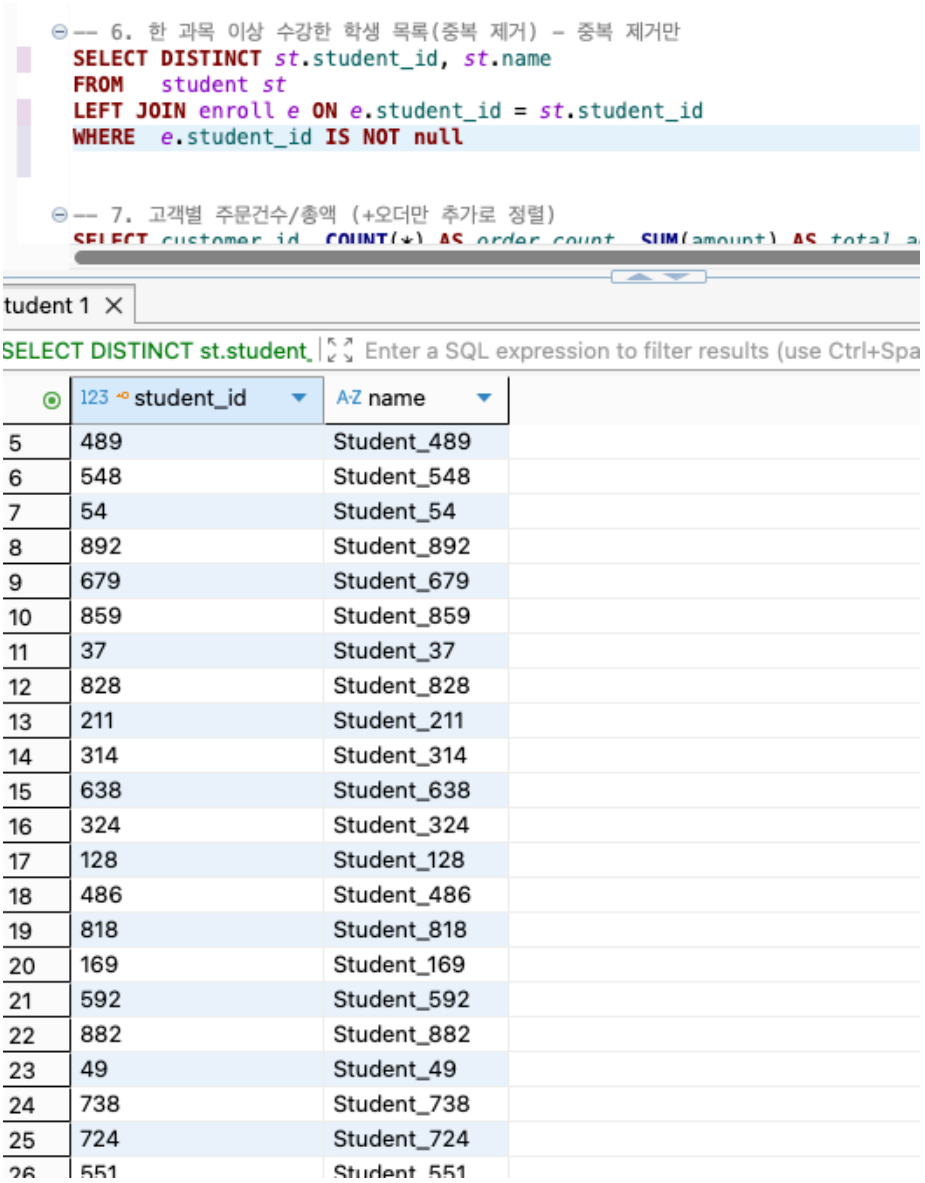
	student_id	name
1	10	Student_10
2	20	Student_20
3	30	Student_30
4	40	Student_40
5	50	Student_50
6	60	Student_60
7	70	Student_70
8	80	Student_80
9	90	Student_90
10	100	Student_100
11	110	Student_110
12	120	Student_120
13	130	Student_130
14	140	Student_140
15	150	Student_150
16	160	Student_160
17	170	Student_170
18	180	Student_180

6. 한 과목 이상 수강한 학생 목록(중복 제거) - 중복 제거만

코드

```sql
SELECT DISTINCT st.student_id, st.name
FROM   student st
LEFT JOIN enroll e ON e.student_id = st.student_id
WHERE  e.student_id IS NOT NULL;
```

결과



```sql
-- 6. 한 과목 이상 수강한 학생 목록(중복 제거) - 중복 제거만
SELECT DISTINCT st.student_id, st.name
FROM   student st
LEFT JOIN enroll e ON e.student_id = st.student_id
WHERE  e.student_id IS NOT null

-- 7. 고객별 주문건수/총액 (+오더만 추가로 정렬)
```

tudent 1 ×

SELECT DISTINCT st.student_ Enter a SQL expression to filter results (use Ctrl+Spa

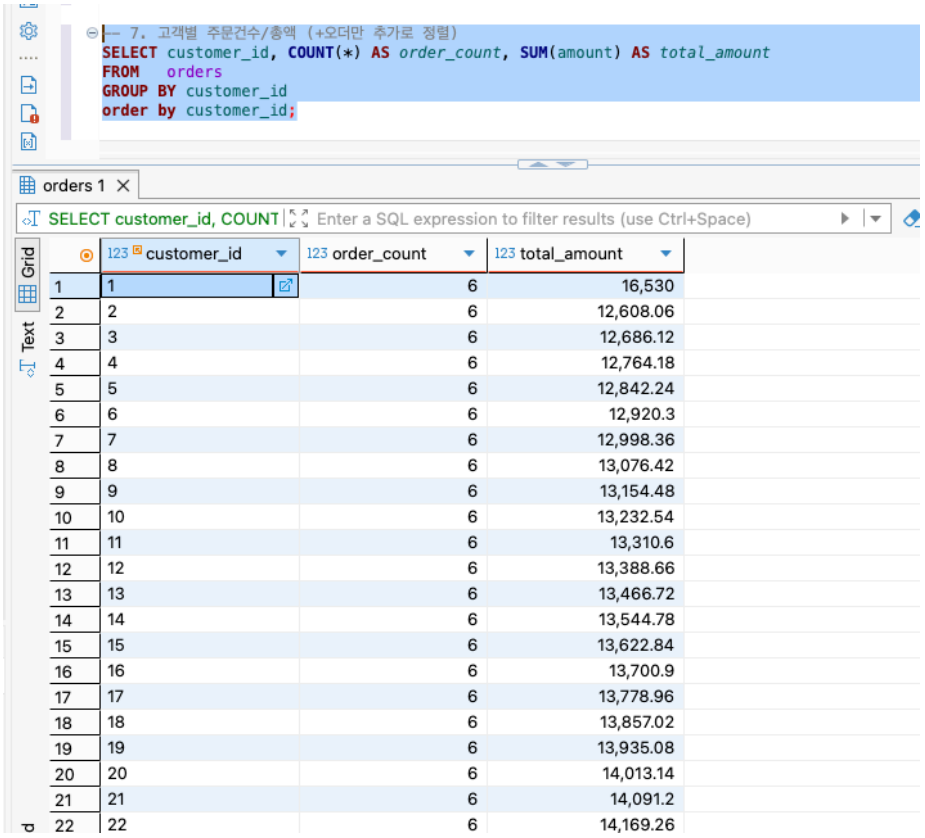
	123 student_id	A-Z name
5	489	Student_489
6	548	Student_548
7	54	Student_54
8	892	Student_892
9	679	Student_679
10	859	Student_859
11	37	Student_37
12	828	Student_828
13	211	Student_211
14	314	Student_314
15	638	Student_638
16	324	Student_324
17	128	Student_128
18	486	Student_486
19	818	Student_818
20	169	Student_169
21	592	Student_592
22	882	Student_882
23	49	Student_49
24	738	Student_738
25	724	Student_724
26	551	Student_551

7. 고객별 주문건수/총액

코드

```sql
SELECT customer_id, COUNT(*) AS order_count, SUM(amount) AS total_amount
FROM   orders
GROUP BY customer_id
order by customer_id; -- db 보기가 불편하여 정렬만 추가했습니다!
```

결과



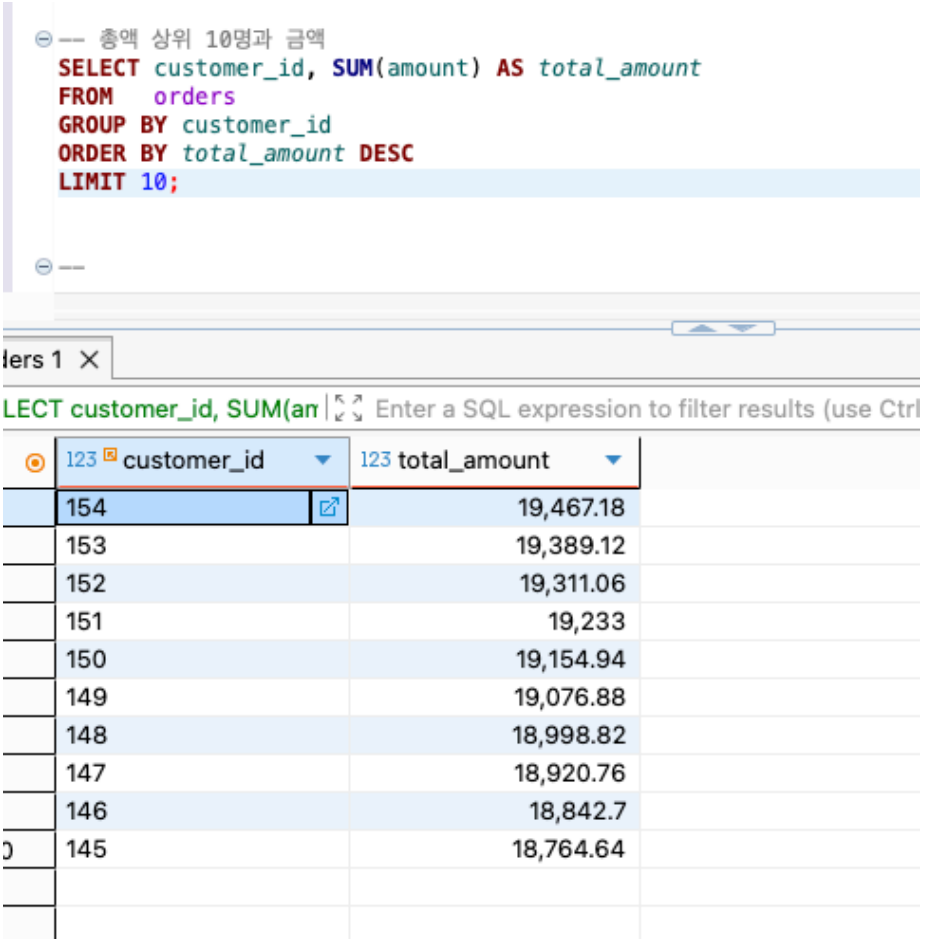
	customer_id	order_count	total_amount
1	1	6	16,530
2	2	6	12,608.06
3	3	6	12,686.12
4	4	6	12,764.18
5	5	6	12,842.24
6	6	6	12,920.3
7	7	6	12,998.36
8	8	6	13,076.42
9	9	6	13,154.48
10	10	6	13,232.54
11	11	6	13,310.6
12	12	6	13,388.66
13	13	6	13,466.72
14	14	6	13,544.78
15	15	6	13,622.84
16	16	6	13,700.9
17	17	6	13,778.96
18	18	6	13,857.02
19	19	6	13,935.08
20	20	6	14,013.14
21	21	6	14,091.2
22	22	6	14,169.26

8. 총액 상위 10명과 금액

코드

```sql
SELECT customer_id, SUM(amount) AS total_amount
FROM   orders
GROUP BY customer_id
ORDER BY total_amount DESC
LIMIT 10;
```

결과

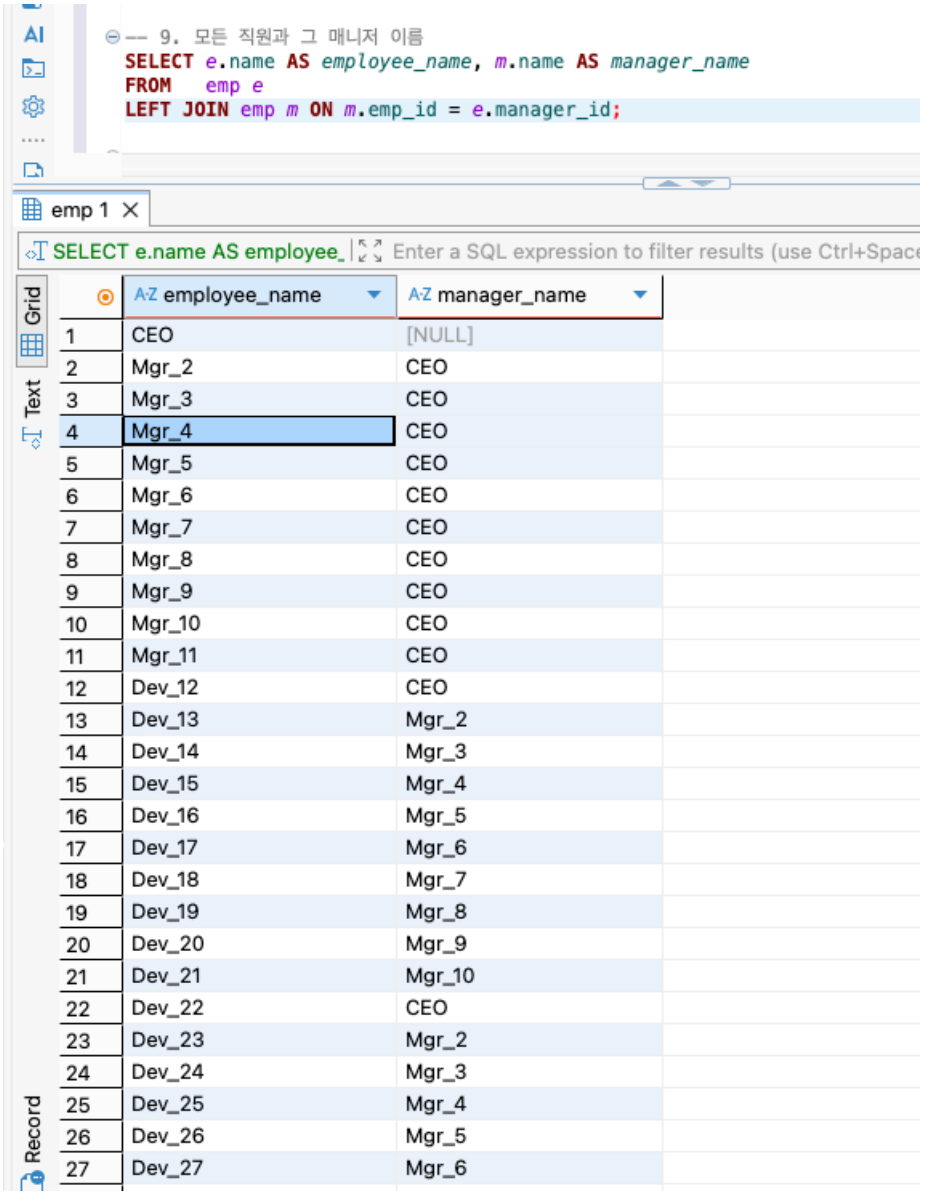


9. 모든 직원과 그 매니저 이름

코드

```sql
SELECT e.name AS employee_name, m.name AS manager_name
FROM   emp e
LEFT JOIN emp m ON m.emp_id = e.manager_id;
```

결과



```sql
-- 9. 모든 직원과 그 매니저 이름
SELECT e.name AS employee_name, m.name AS manager_name
FROM   emp e
LEFT JOIN emp m ON m.emp_id = e.manager_id;
```

emp 1

SELECT e.name AS employee_ | Enter a SQL expression to filter results (use Ctrl+Space

Grid | Text | Record

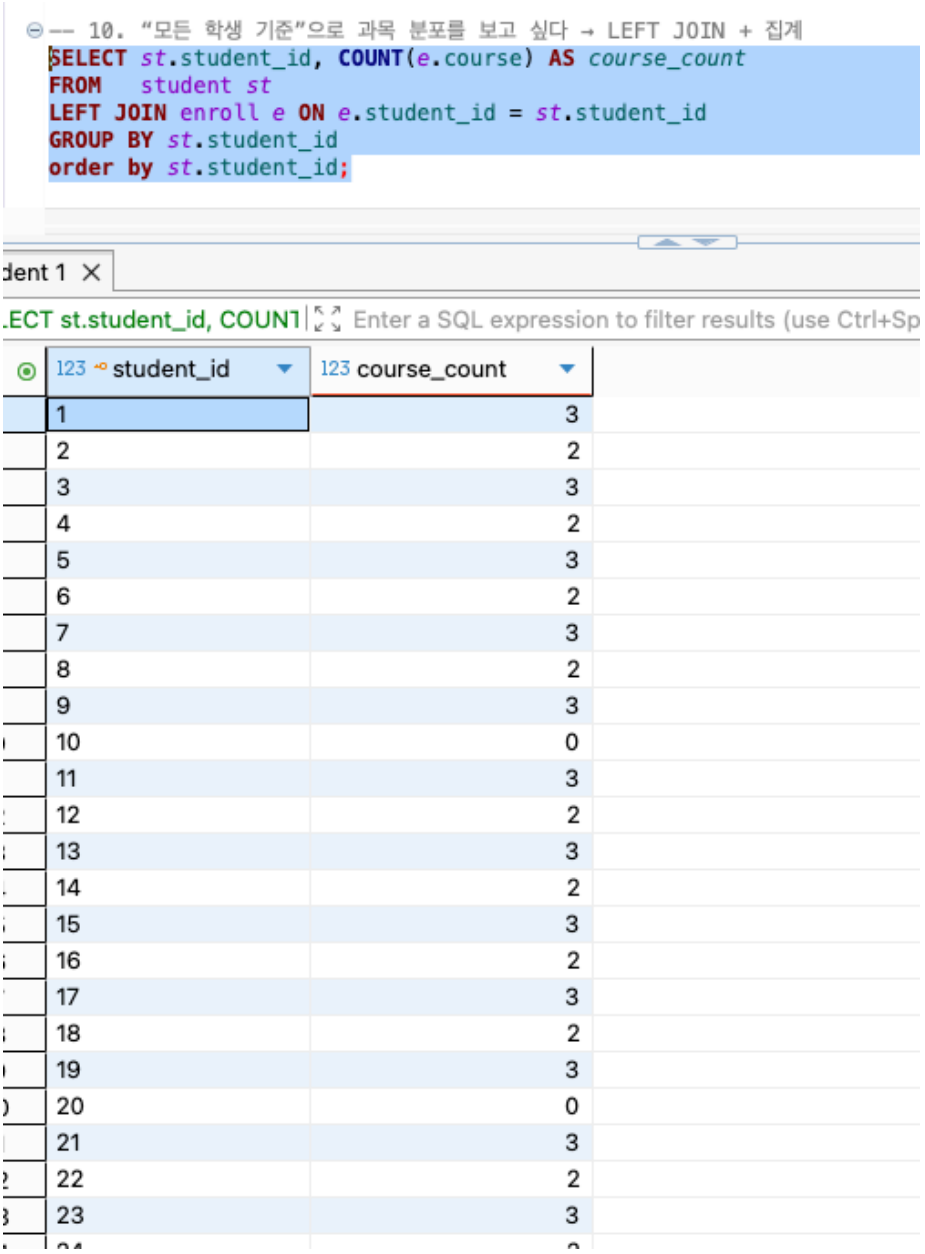
	employee_name	manager_name
1	CEO	[NULL]
2	Mgr_2	CEO
3	Mgr_3	CEO
4	Mgr_4	CEO
5	Mgr_5	CEO
6	Mgr_6	CEO
7	Mgr_7	CEO
8	Mgr_8	CEO
9	Mgr_9	CEO
10	Mgr_10	CEO
11	Mgr_11	CEO
12	Dev_12	CEO
13	Dev_13	Mgr_2
14	Dev_14	Mgr_3
15	Dev_15	Mgr_4
16	Dev_16	Mgr_5
17	Dev_17	Mgr_6
18	Dev_18	Mgr_7
19	Dev_19	Mgr_8
20	Dev_20	Mgr_9
21	Dev_21	Mgr_10
22	Dev_22	CEO
23	Dev_23	Mgr_2
24	Dev_24	Mgr_3
25	Dev_25	Mgr_4
26	Dev_26	Mgr_5
27	Dev_27	Mgr_6

10. “모든 학생 기준”으로 과목 분포를 보고 싶다 → LEFT JOIN + 집계

코드

```sql
SELECT st.student_id, COUNT(e.course) AS course_count
FROM   student st
LEFT JOIN enroll e ON e.student_id = st.student_id
GROUP BY st.student_id
order by st.student_id; -- 정렬만 추가했습니다!
```

결과



```sql
-- 10. "모든 학생 기준"으로 과목 분포를 보고 싶다 → LEFT JOIN + 집계
SELECT st.student_id, COUNT(e.course) AS course_count
FROM   student st
LEFT JOIN enroll e ON e.student_id = st.student_id
GROUP BY st.student_id
order by st.student_id;
```

dent 1 ×

.ECT st.student_id, COUN1 | Enter a SQL expression to filter results (use Ctrl+Sp

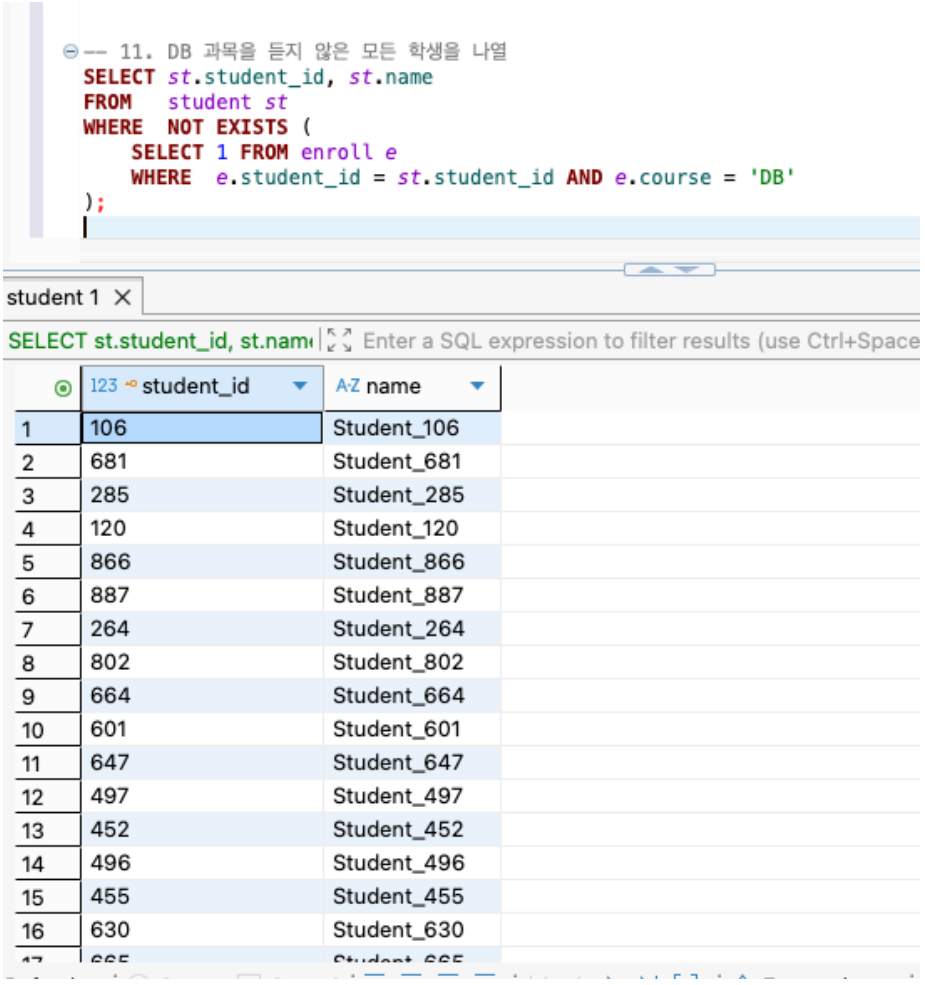
123 student_id	123 course_count
1	3
2	2
3	3
4	2
5	3
6	2
7	3
8	2
9	3
10	0
11	3
12	2
13	3
14	2
15	3
16	2
17	3
18	2
19	3
20	0
21	3
22	2
23	3

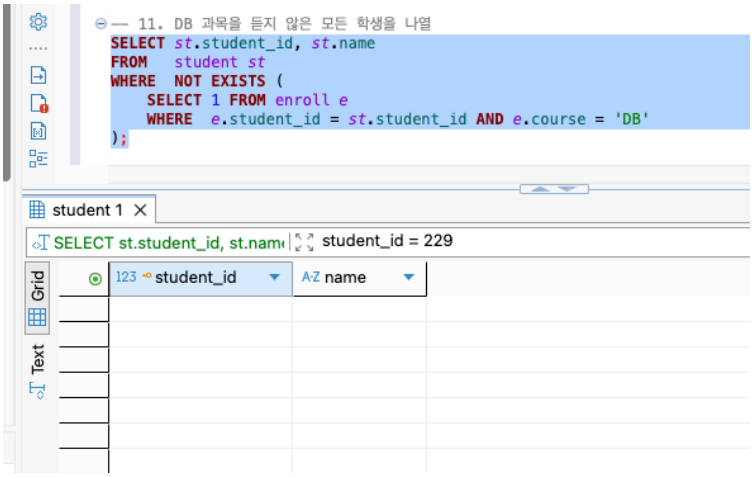
11. DB 과목을 듣지 않은 모든 학생을 나열

코드

```sql
SELECT st.student_id, st.name
FROM   student st
WHERE  NOT EXISTS (
    SELECT 1 FROM enroll e
    WHERE  e.student_id = st.student_id AND e.course = 'DB'
);
```

결과





→ DB 를 수강하는 229 번 학생이 나오지 않음을 확인

12. (가정) 과목별로 매니저가 운영 책임을 갖는다고 가정하고,emp의 매니저(이름 'Mgr_'로 시작)와 과목을 임의로 매핑한 테이블 course_owner(course, manager_id)를 만든 뒤,과목별 수강 인원 + 책임 매니저 이름 리포트를 작성하세요

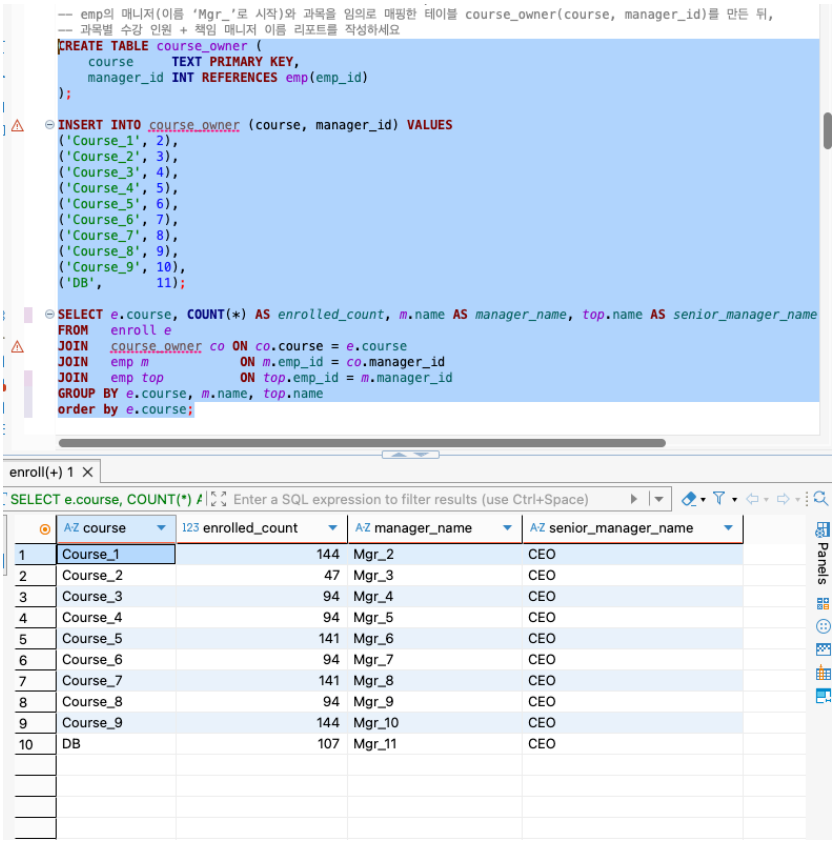
코드

```sql
CREATE TABLE course_owner (
    course     TEXT PRIMARY KEY,
    manager_id INT REFERENCES emp(emp_id)
);

INSERT INTO course_owner (course, manager_id) VALUES
('Course_1', 2),
('Course_2', 3),
('Course_3', 4),
('Course_4', 5),
('Course_5', 6),
('Course_6', 7),
('Course_7', 8),
('Course_8', 9),
('Course_9', 10),
('DB',       11);

SELECT e.course, COUNT(*) AS enrolled_count, m.name AS manager_name, top.name AS seni
or_manager_name
FROM   enroll e
JOIN   course_owner co ON co.course = e.course
JOIN   emp m             ON m.emp_id = co.manager_id
JOIN   emp top           ON top.emp_id = m.manager_id
GROUP BY e.course, m.name, top.name
order by e.course; -- orderby 만 요구사항 외 추가했습니다!
```

결과

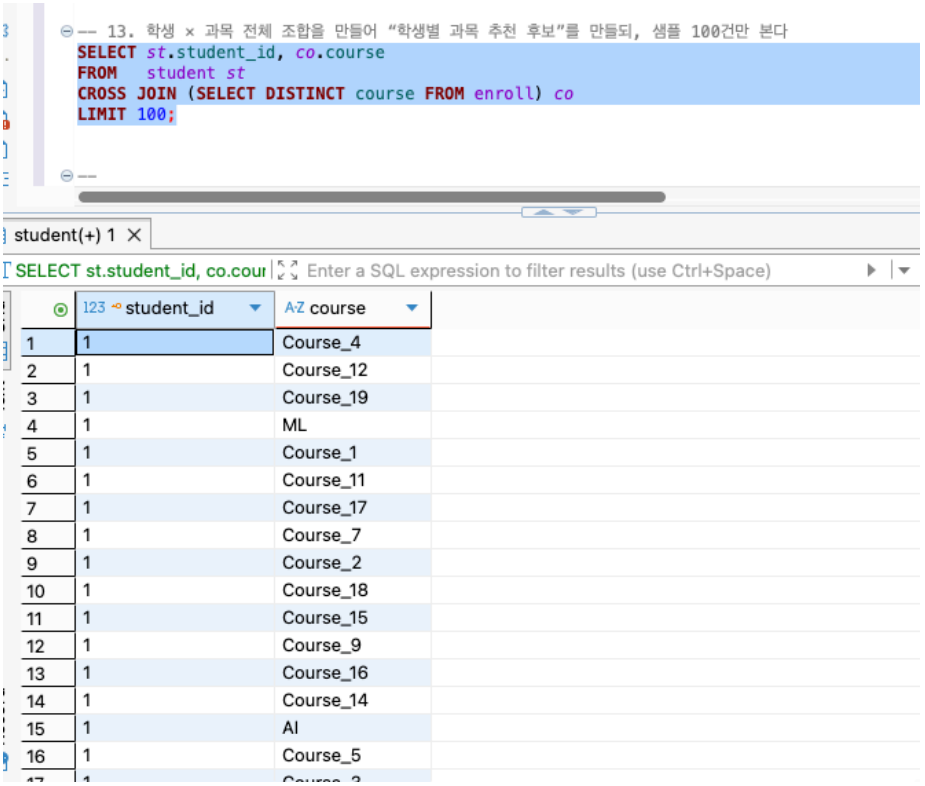


13. 학생 × 과목 전체 조합을 만들어 “학생별 과목 추천 후보”를 만들되, 샘플 100건만 본다

코드

```sql
SELECT st.student_id, co.course
FROM   student st
CROSS JOIN (SELECT DISTINCT course FROM enroll) co
LIMIT 100;
```

결과



```sql
-- 13. 학생 × 과목 전체 조합을 만들어 “학생별 과목 추천 후보”를 만들되, 샘플 100건만 본다
SELECT st.student_id, co.course
FROM   student st
CROSS JOIN (SELECT DISTINCT course FROM enroll) co
LIMIT 100;
```

student(+) 1

SELECT st.student_id, co.cour | Enter a SQL expression to filter results (use Ctrl+Space)

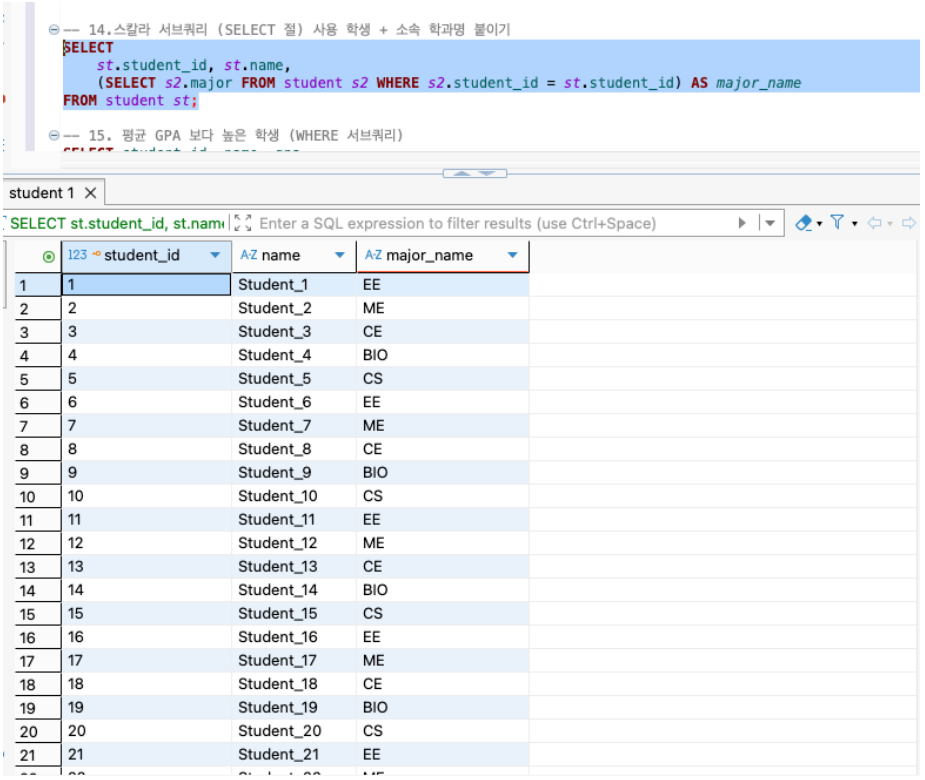
	student_id	course
1	1	Course_4
2	1	Course_12
3	1	Course_19
4	1	ML
5	1	Course_1
6	1	Course_11
7	1	Course_17
8	1	Course_7
9	1	Course_2
10	1	Course_18
11	1	Course_15
12	1	Course_9
13	1	Course_16
14	1	Course_14
15	1	AI
16	1	Course_5

14. 스칼라 서브쿼리 (SELECT 절) 사용 학생 + 소속 학과명 붙이기

코드

```
SELECT
    st.student_id, st.name,
    (SELECT s2.major FROM student s2 WHERE s2.student_id = st.student_id) AS major_na
me
FROM student st;
```

결과



```
-- 14.스칼라 서브쿼리 (SELECT 절) 사용 학생 + 소속 학과명 붙이기
SELECT
    st.student_id, st.name,
    (SELECT s2.major FROM student s2 WHERE s2.student_id = st.student_id) AS major_name
FROM student st;

-- 15. 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)
```

student 1

SELECT st.student_id, st.nam… | Enter a SQL expression to filter results (use Ctrl+Space)

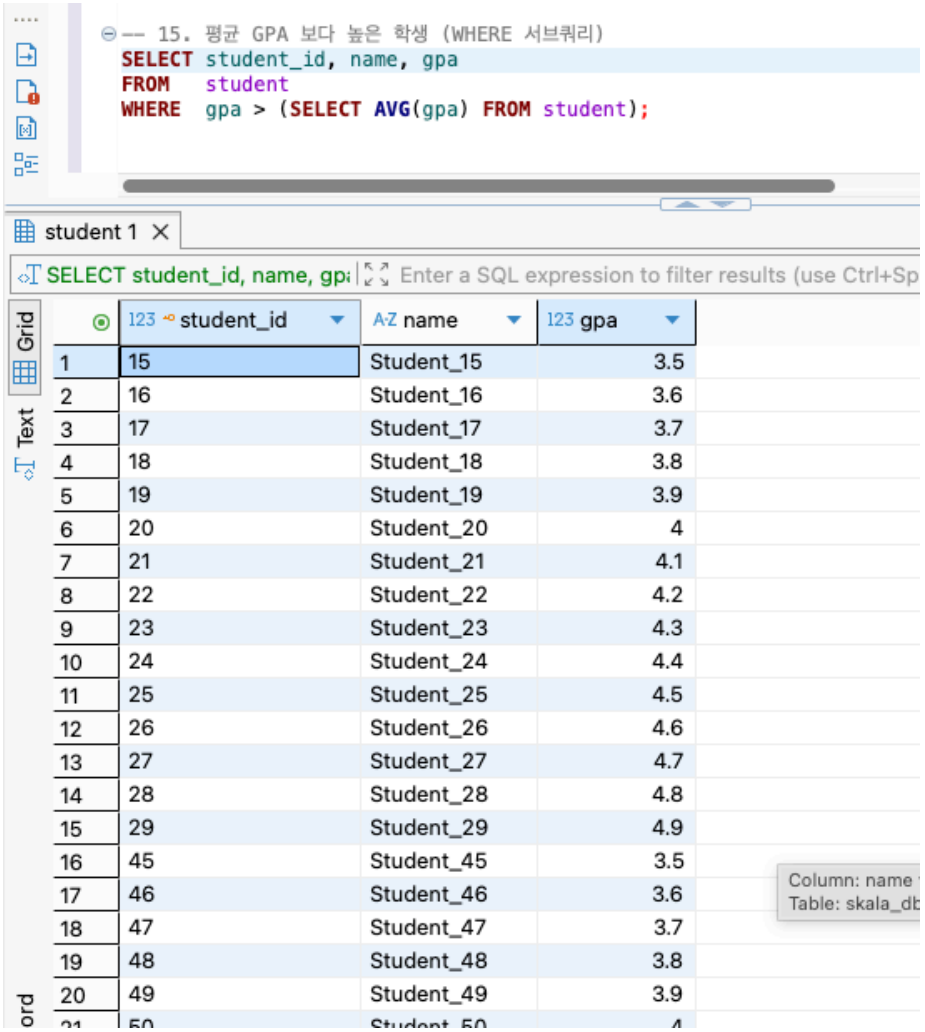
	student_id	name	major_name
1	1	Student_1	EE
2	2	Student_2	ME
3	3	Student_3	CE
4	4	Student_4	BIO
5	5	Student_5	CS
6	6	Student_6	EE
7	7	Student_7	ME
8	8	Student_8	CE
9	9	Student_9	BIO
10	10	Student_10	CS
11	11	Student_11	EE
12	12	Student_12	ME
13	13	Student_13	CE
14	14	Student_14	BIO
15	15	Student_15	CS
16	16	Student_16	EE
17	17	Student_17	ME
18	18	Student_18	CE
19	19	Student_19	BIO
20	20	Student_20	CS
21	21	Student_21	EE

15. 평균 GPA 보다 높은 학생 (WHERE 서브쿼리)

코드

```sql
SELECT student_id, name, gpa
FROM   student
WHERE  gpa > (SELECT AVG(gpa) FROM student);
```

결과

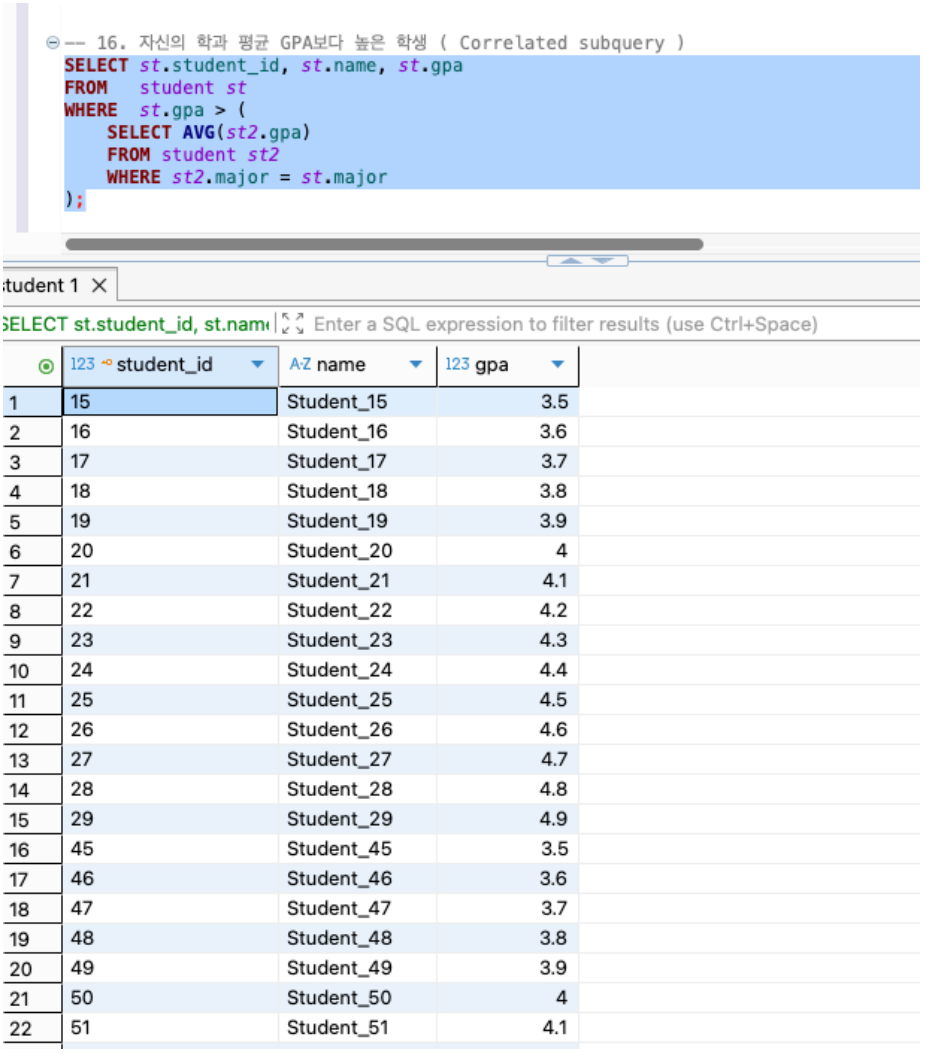


16. 자신의 학과 평균 GPA보다 높은 학생 (Correlated subquery)

코드

```sql
SELECT st.student_id, st.name, st.gpa
FROM   student st
WHERE  st.gpa > (
    SELECT AVG(st2.gpa)
    FROM student st2
    WHERE st2.major = st.major
);
```

결과



```sql
-- 16. 자신의 학과 평균 GPA보다 높은 학생 ( Correlated subquery )
SELECT st.student_id, st.name, st.gpa
FROM   student st
WHERE  st.gpa > (
    SELECT AVG(st2.gpa)
    FROM student st2
    WHERE st2.major = st.major
);
```

tudent 1 ×

ELECT st.student_id, st.nam Enter a SQL expression to filter results (use Ctrl+Space)

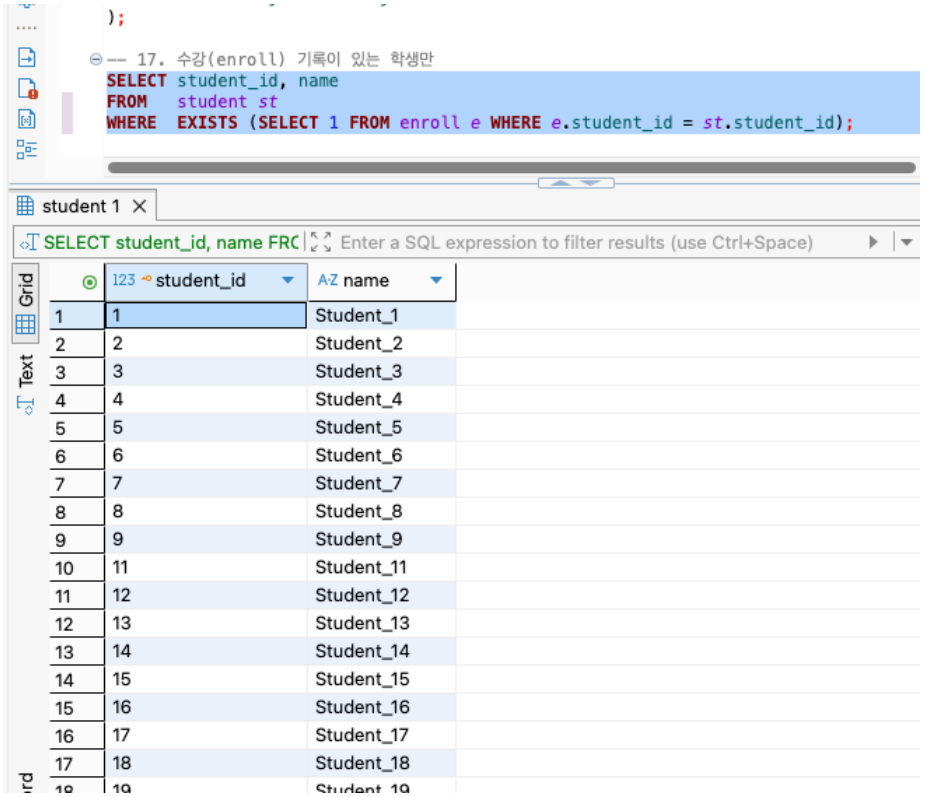
	student_id	name	gpa
1	15	Student_15	3.5
2	16	Student_16	3.6
3	17	Student_17	3.7
4	18	Student_18	3.8
5	19	Student_19	3.9
6	20	Student_20	4
7	21	Student_21	4.1
8	22	Student_22	4.2
9	23	Student_23	4.3
10	24	Student_24	4.4
11	25	Student_25	4.5
12	26	Student_26	4.6
13	27	Student_27	4.7
14	28	Student_28	4.8
15	29	Student_29	4.9
16	45	Student_45	3.5
17	46	Student_46	3.6
18	47	Student_47	3.7
19	48	Student_48	3.8
20	49	Student_49	3.9
21	50	Student_50	4
22	51	Student_51	4.1

17. 수강(enroll) 기록이 있는 학생만

코드

```sql
SELECT student_id, name
FROM   student st
WHERE  EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = st.student_id);
```

결과

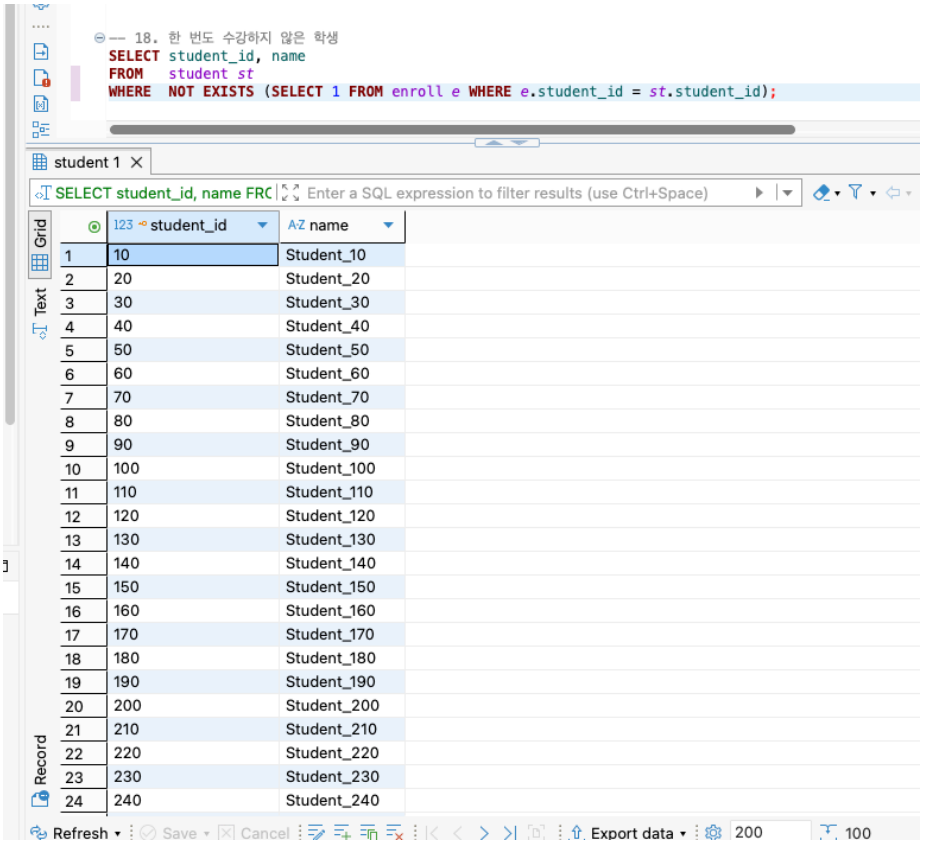


18. 한 번도 수강하지 않은 학생

코드

```sql
SELECT student_id, name
FROM   student st
WHERE  NOT EXISTS (SELECT 1 FROM enroll e WHERE e.student_id = st.student_id);
```

결과



19. HR 학과 학생 일부와의 비교 데모

요구조건

`lab.student` 테이블에는 `student_id 981~1000` 번 학생들이 `major = 'HR'` 로 지정되어 있습니다. 이 학생들 중 일부를 골라, 다른 전공 학생과 GPA를 비교하는 쿼리를 작성하시면 됩니다.

요구사항

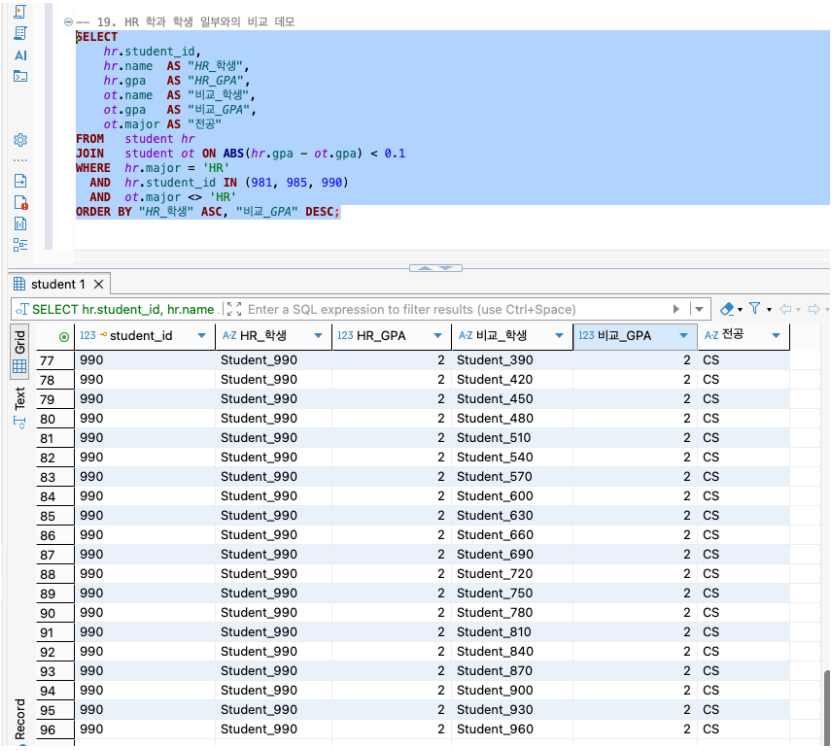
1. `lab.student` 테이블을 자기 자신과 조인(Self Join)하도록 작성하세요. 이때 두 참조에 서로 다른 별칭(alias)을 붙이세요.
2. 한쪽은 `major = 'HR'` 인 학생만, 다른 한쪽은 `major` 가 `'HR'` 이 아닌 학생만 나오도록 조건을 거세요.
3. 두 학생의 GPA 차이가 0.1 미만인 경우만 결과에 포함되도록 조인 조건을 추가하세요. (`=` 이 아닌 비교 연산자를 사용해야 합니다.)
4. HR 학생은 전체가 아니라 `student_id` 가 981, 985, 990인 세 명만 비교 대상으로 삼으세요.
5. 결과에는 다음 6개 컬럼이 순서대로 나오도록 컬럼 별칭(AS)을 지정하세요:

   `HR_학생` , `HR_GPA` , `비교_학생` , `비교_GPA` , `전공`
6. 결과를 `HR_학생` 오름차순, `비교_GPA` 내림차순으로 정렬하세요.

코드

```sql
SELECT
    hr.student_id,
    hr.name  AS "HR_학생",
    hr.gpa   AS "HR_GPA",
    ot.name  AS "비교_학생",
    ot.gpa   AS "비교_GPA",
    ot.major AS "전공"
FROM   student hr
JOIN   student ot ON ABS(hr.gpa - ot.gpa) < 0.1
WHERE  hr.major = 'HR'
  AND  hr.student_id IN (981, 985, 990)
  AND  ot.major <> 'HR'
ORDER BY "HR_학생" ASC, "비교_GPA" DESC;
```

결과



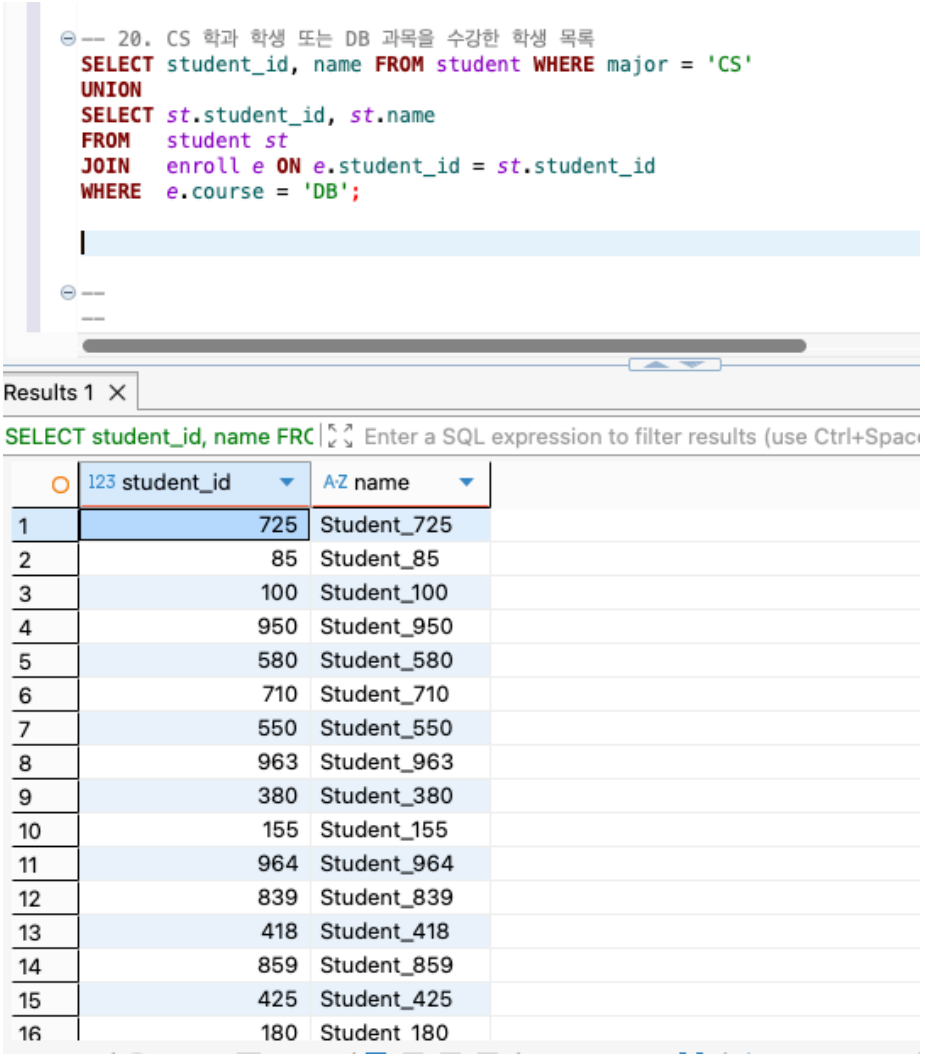
	student_id	HR_학생	HR_GPA	비교_학생	비교_GPA	전공
77	990	Student_990	2	Student_390	2	CS
78	990	Student_990	2	Student_420	2	CS
79	990	Student_990	2	Student_450	2	CS
80	990	Student_990	2	Student_480	2	CS
81	990	Student_990	2	Student_510	2	CS
82	990	Student_990	2	Student_540	2	CS
83	990	Student_990	2	Student_570	2	CS
84	990	Student_990	2	Student_600	2	CS
85	990	Student_990	2	Student_630	2	CS
86	990	Student_990	2	Student_660	2	CS
87	990	Student_990	2	Student_690	2	CS
88	990	Student_990	2	Student_720	2	CS
89	990	Student_990	2	Student_750	2	CS
90	990	Student_990	2	Student_780	2	CS
91	990	Student_990	2	Student_810	2	CS
92	990	Student_990	2	Student_840	2	CS
93	990	Student_990	2	Student_870	2	CS
94	990	Student_990	2	Student_900	2	CS
95	990	Student_990	2	Student_930	2	CS
96	990	Student_990	2	Student_960	2	CS

20. CS 학과 학생 또는 DB 과목을 수강한 학생 목록

코드

```sql
SELECT student_id, name FROM student WHERE major = 'CS'
UNION
SELECT st.student_id, st.name
FROM   student st
JOIN   enroll e ON e.student_id = st.student_id
WHERE  e.course = 'DB';
```

결과



	123 student_id	A-Z name
1	725	Student_725
2	85	Student_85
3	100	Student_100
4	950	Student_950
5	580	Student_580
6	710	Student_710
7	550	Student_550
8	963	Student_963
9	380	Student_380
10	155	Student_155
11	964	Student_964
12	839	Student_839
13	418	Student_418
14	859	Student_859
15	425	Student_425
16	180	Student_180

21. student 테이블에는 학과(major)와 GPA가 있습니다. 학과별·GPA 구간별 인원을 집계하되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력

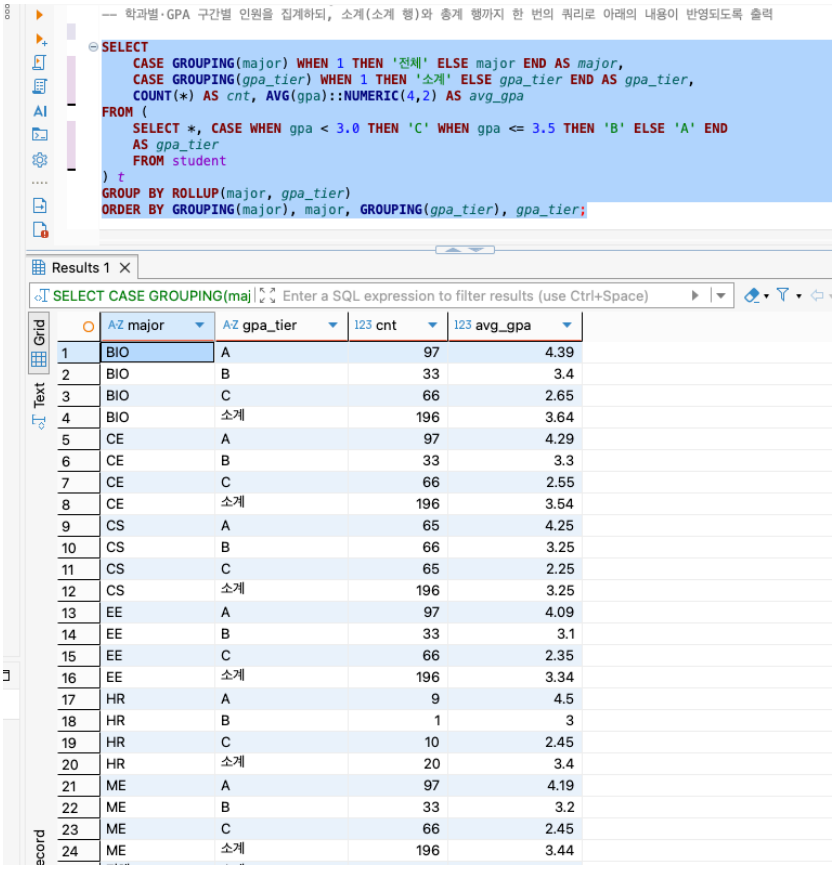
요구조건

- GPA를 구간 (3.0 미만 / 3.0~3.5 / 3.5 초과)으로 분류하는 파생 컬럼을 추가
- GROUP BY ROLLUP(major, gpa_tier)로 학과별, 전체 소계를 동시에 조회
- GROUPING(major) 함수로 소계 행에 '전체' 라벨을 붙일 것
- 결과를 major, gpa_tier 순으로 정렬하되 소계 행은 하단에 표시

코드

```sql
SELECT
    CASE GROUPING(major) WHEN 1 THEN '전체' ELSE major END AS major,
    CASE GROUPING(gpa_tier) WHEN 1 THEN '소계' ELSE gpa_tier END AS gpa_tier,
    COUNT(*) AS cnt, AVG(gpa)::NUMERIC(4,2) AS avg_gpa
FROM (
    SELECT *, CASE WHEN gpa < 3.0 THEN 'C' WHEN gpa <= 3.5 THEN 'B' ELSE 'A' END
    AS gpa_tier
    FROM student
) t
GROUP BY ROLLUP(major, gpa_tier)
ORDER BY GROUPING(major), major, GROUPING(gpa_tier), gpa_tier;
```

결과



-- 학과별·GPA 구간별 인원을 집계하되, 소계(소계 행)와 총계 행까지 한 번의 쿼리로 아래의 내용이 반영되도록 출력

Results 1

SELECT CASE GROUPING(maj | Enter a SQL expression to filter results (use Ctrl+Space)

	major	gpa_tier	cnt	avg_gpa
1	BIO	A	97	4.39
2	BIO	B	33	3.4
3	BIO	C	66	2.65
4	BIO	소계	196	3.64
5	CE	A	97	4.29
6	CE	B	33	3.3
7	CE	C	66	2.55
8	CE	소계	196	3.54
9	CS	A	65	4.25
10	CS	B	66	3.25
11	CS	C	65	2.25
12	CS	소계	196	3.25
13	EE	A	97	4.09
14	EE	B	33	3.1
15	EE	C	66	2.35
16	EE	소계	196	3.34
17	HR	A	9	4.5
18	HR	B	1	3
19	HR	C	10	2.45
20	HR	소계	20	3.4
21	ME	A	97	4.19
22	ME	B	33	3.2
23	ME	C	66	2.45
24	ME	소계	196	3.44

→ 각 메이저 별 합을 소계로 표현, 모든 count 합 을 전체로 표현

22. emp 테이블은 CEO(manager_id = NULL) → 매니저(10명) → 개발자(300명)의 3단계 계층 구조입니다. WITH RECURSIVE로 모든 직원의 계층 경로와 깊이를 출력 - 그룹없을 때 널 값이 나오지 않도록 전체 소계

요구조건

- WITH RECURSIVE로 CEO에서 시작하는 조직 트리를 탐색
- 각 행에 depth(0=CEO), path 컬럼을 포함 (예: 'CEO > Mgr_2 > Dev_15')
- 매니저별 직속 부하 직원 수를 집계하는 쿼리 별도로 작성 (컬럼명 : direct_reports)

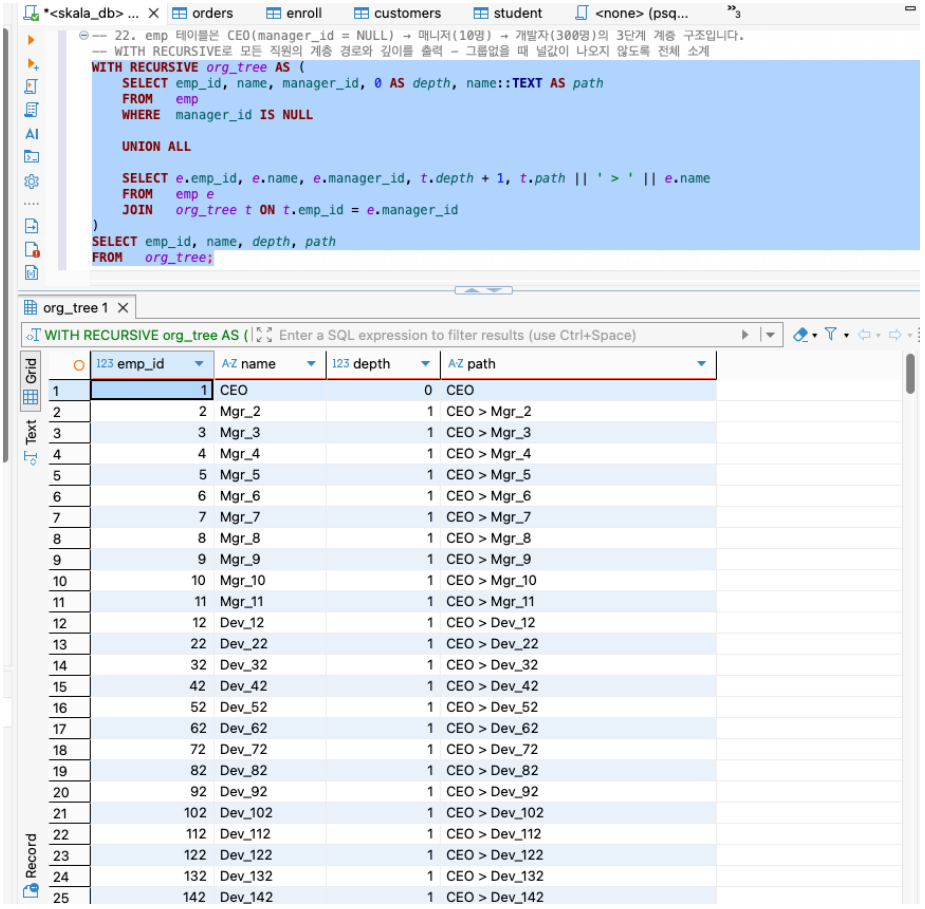
코드

```sql
WITH RECURSIVE org_tree AS (
    SELECT emp_id, name, manager_id, 0 AS depth, name::TEXT AS path
    FROM   emp
    WHERE  manager_id IS NULL

    UNION ALL

    SELECT e.emp_id, e.name, e.manager_id, t.depth + 1, t.path || ' > ' || e.name
    FROM   emp e
    JOIN   org_tree t ON t.emp_id = e.manager_id
)
SELECT emp_id, name, depth, path
FROM   org_tree;
```

결과



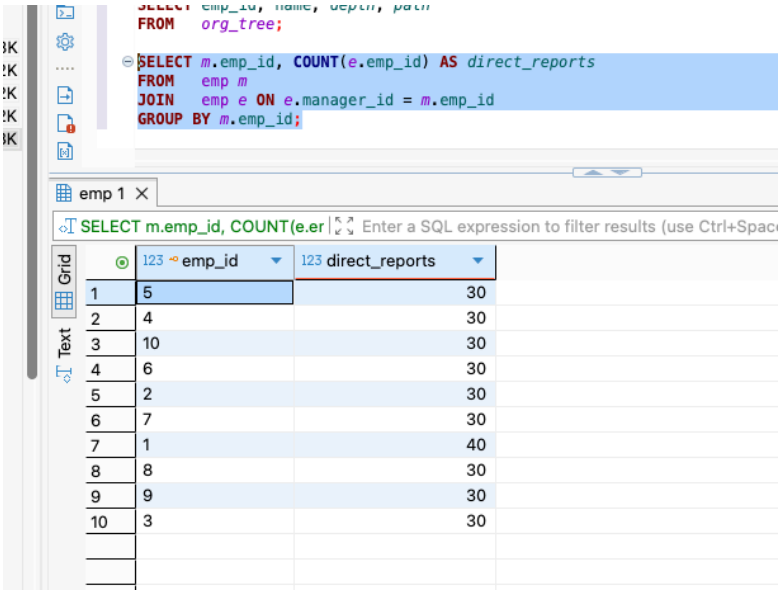
emp_id	name	depth	path
1	CEO	0	CEO
2	Mgr_2	1	CEO > Mgr_2
3	Mgr_3	1	CEO > Mgr_3
4	Mgr_4	1	CEO > Mgr_4
5	Mgr_5	1	CEO > Mgr_5
6	Mgr_6	1	CEO > Mgr_6
7	Mgr_7	1	CEO > Mgr_7
8	Mgr_8	1	CEO > Mgr_8
9	Mgr_9	1	CEO > Mgr_9
10	Mgr_10	1	CEO > Mgr_10
11	Mgr_11	1	CEO > Mgr_11
12	Dev_12	1	CEO > Dev_12
22	Dev_22	1	CEO > Dev_22
32	Dev_32	1	CEO > Dev_32
42	Dev_42	1	CEO > Dev_42
52	Dev_52	1	CEO > Dev_52
62	Dev_62	1	CEO > Dev_62
72	Dev_72	1	CEO > Dev_72
82	Dev_82	1	CEO > Dev_82
92	Dev_92	1	CEO > Dev_92
102	Dev_102	1	CEO > Dev_102
112	Dev_112	1	CEO > Dev_112
122	Dev_122	1	CEO > Dev_122
132	Dev_132	1	CEO > Dev_132
142	Dev_142	1	CEO > Dev_142

매니저별 직속 부하 직원 수를 집계하는 쿼리 별도로 작성 (컬럼명 : direct_reports)

코드

```sql
-- 매니저별 직속 부하 직원 수를 집계하는 쿼리 별도로 작성 (컬럼명 : direct_reports)
SELECT m.emp_id, COUNT(e.emp_id) AS direct_reports
FROM   emp m
JOIN   emp e ON e.manager_id = m.emp_id
GROUP BY m.emp_id;
```

결과



23. 각 학과(major)별로 GPA가 높은 순서로 순위를 매기고, 상위 3명씩만 추출하는 쿼리를 Window Function으로 작성 (서브쿼리 방식과 CTE 방식 모두 작성)

요구조건

- 학과 내 순위를 계산 ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC)
- GPA가 동일한 경우 student_id 오름차순을 2차 기준으로 사용
- RANK()와 DENSE_RANK()를 함께 계산해 동점 처리 방식 차이를 비교
- 결과에 전체 학과별 학생 수(total_in_major)도 COUNT() OVER(PARTITION BY major)로 추가

코드

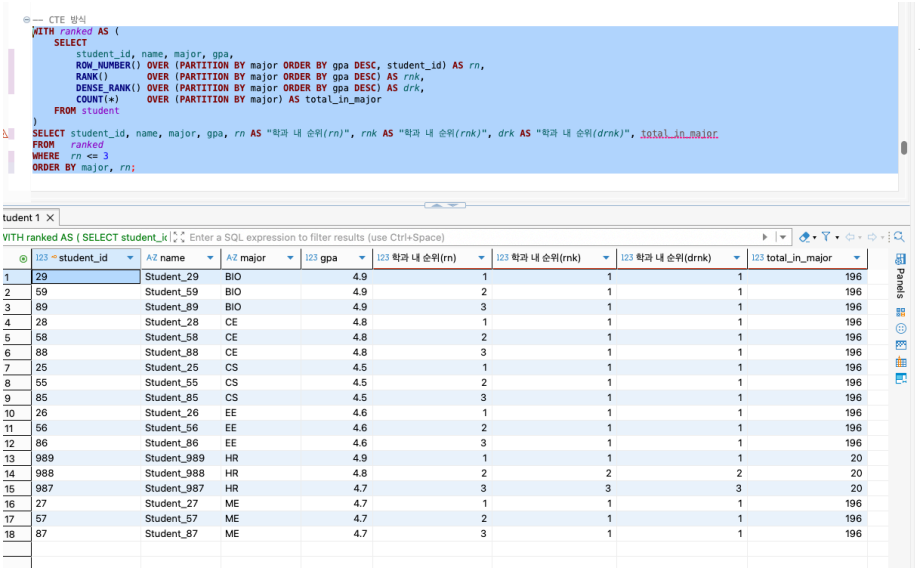
서브쿼리 방식

```sql
SELECT student_id, major, gpa, rn AS "학과 내 순위(rn)", rnk AS "학과 내 순위(rnk)", drn
k AS "학과 내 순위(drnk)", total_in_major
FROM (
    SELECT
        student_id, major, gpa,
        ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC) AS rn,
        RANK()       OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
        DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drnk,
        COUNT(*)     OVER (PARTITION BY major) AS total_in_major
    FROM student
) ranked
WHERE rn <= 3;
```

CTE 방식

```sql
WITH ranked AS (
    SELECT
        student_id, name, major, gpa,
        ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
        RANK()       OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
        DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drk,
        COUNT(*)     OVER (PARTITION BY major) AS total_in_major
    FROM student
)
SELECT student_id, name, major, gpa, rn AS "학과 내 순위(rn)", rnk AS "학과 내 순위(rn
k)", drk AS "학과 내 순위(drnk)", total_in_major
FROM   ranked
WHERE  rn <= 3
ORDER BY major, rn;
```

결과



-- CTE 방식

```sql
WITH ranked AS (
    SELECT
        student_id, name, major, gpa,
        ROW_NUMBER() OVER (PARTITION BY major ORDER BY gpa DESC, student_id) AS rn,
        RANK()       OVER (PARTITION BY major ORDER BY gpa DESC) AS rnk,
        DENSE_RANK() OVER (PARTITION BY major ORDER BY gpa DESC) AS drk,
        COUNT(*)     OVER (PARTITION BY major) AS total_in_major
    FROM student
)
SELECT student_id, name, major, gpa, rn AS "학과 내 순위(rn)", rnk AS "학과 내 순위(rnk)", drk AS "학과 내 순위(drnk)", total_in_major
FROM   ranked
WHERE  rn <= 3
ORDER BY major, rn;
```

tudent 1 ×

VITH ranked AS (SELECT student_ic | Enter a SQL expression to filter results (use Ctrl+Space)

	123 student_id	AZ name	AZ major	123 gpa	123 학과 내 순위(rn)	123 학과 내 순위(rnk)	123 학과 내 순위(drnk)	123 total_in_major
1	29	Student_29	BIO	4.9	1	1	1	196
2	59	Student_59	BIO	4.9	2	1	1	196
3	89	Student_89	BIO	4.9	3	1	1	196
4	28	Student_28	CE	4.8	1	1	1	196
5	58	Student_58	CE	4.8	2	1	1	196
6	88	Student_88	CE	4.8	3	1	1	196
7	25	Student_25	CS	4.5	1	1	1	196
8	55	Student_55	CS	4.5	2	1	1	196
9	85	Student_85	CS	4.5	3	1	1	196
10	26	Student_26	EE	4.6	1	1	1	196
11	56	Student_56	EE	4.6	2	1	1	196
12	86	Student_86	EE	4.6	3	1	1	196
13	989	Student_989	HR	4.9	1	1	1	20
14	988	Student_988	HR	4.8	2	2	2	20
15	987	Student_987	HR	4.7	3	3	3	20
16	27	Student_27	ME	4.7	1	1	1	196
17	57	Student_57	ME	4.7	2	1	1	196
18	87	Student_87	ME	4.7	3	1	1	196

Panels

24. enroll 테이블을 student_id 기준으로 정렬하고, 각 학생의 이전 수강 과목 대비 성적 변화를 LAG()로 계산 (성적은 A=4, B=3, C=2, D=1 점수로 변환)

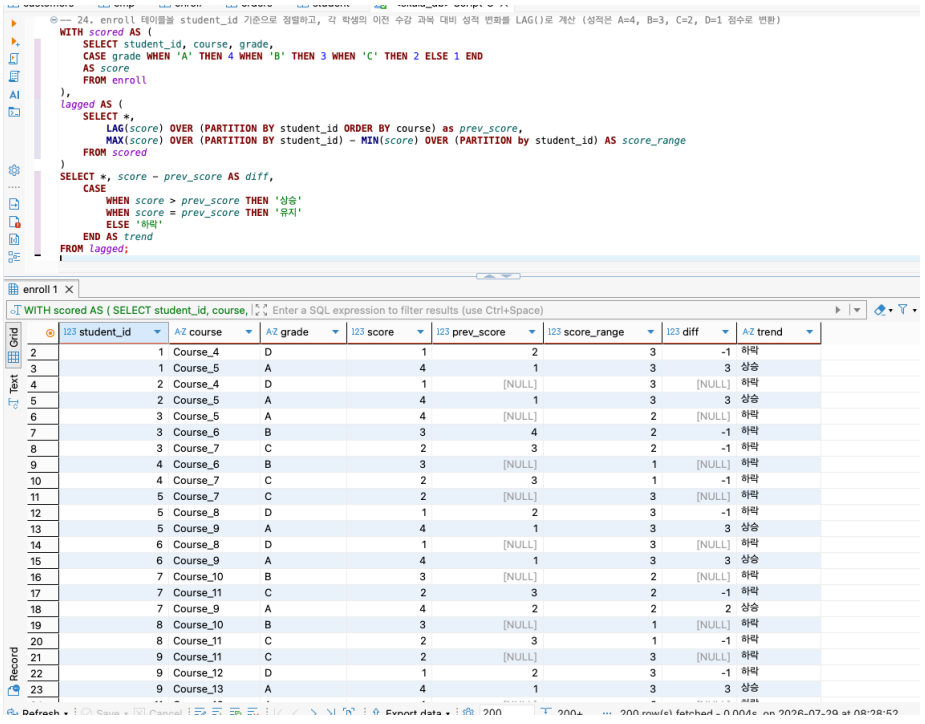
요구조건

- grade를 숫자 점수(A=4, B=3, C=2, D=1)로 변환하는 CASE 식을 작성
- LAG(score) OVER (PARTITION BY student_id ORDER BY course)로 이전 과목 점수를 가져올 것
- 현재 점수 - 이전 점수를 diff 컬럼으로 추가하고, 상승/유지/하락을 텍스트로 표시
- 학생별 최고점과 최저점의 차이(score_range)를 Window Function으로 계산

코드

```
WITH scored AS (
    SELECT student_id, course, grade,
    CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3 WHEN 'C' THEN 2 ELSE 1 END
    AS score
    FROM enroll
),
lagged AS (
    SELECT *,
        LAG(score) OVER (PARTITION BY student_id ORDER BY course) as prev_score,
        MAX(score) OVER (PARTITION BY student_id) - MIN(score) OVER (PARTITION by stu
dent_id) AS score_range
    FROM scored
)
SELECT *, score - prev_score AS diff,
    CASE
        WHEN score > prev_score THEN '상승'
        WHEN score = prev_score THEN '유지'
        ELSE '하락'
    END AS trend
FROM lagged;
```

결과



	student_id	course	grade	score	prev_score	score_range	diff	trend
2	1	Course_4	D	1	2	3	-1	하락
3	1	Course_5	A	4	1	3	3	상승
4	2	Course_4	D	1	[NULL]	3	[NULL]	하락
5	2	Course_5	A	4	1	3	3	상승
6	3	Course_5	A	4	[NULL]	2	[NULL]	하락
7	3	Course_6	B	3	4	2	-1	하락
8	3	Course_7	C	2	3	2	-1	하락
9	4	Course_6	B	3	[NULL]	1	[NULL]	하락
10	4	Course_7	C	2	3	1	-1	하락
11	5	Course_7	C	2	[NULL]	3	[NULL]	하락
12	5	Course_8	D	1	2	3	-1	하락
13	5	Course_9	A	4	1	3	3	상승
14	6	Course_8	D	1	[NULL]	3	[NULL]	하락
15	6	Course_9	A	4	1	3	3	상승
16	7	Course_10	B	3	[NULL]	2	[NULL]	하락
17	7	Course_11	C	2	3	2	-1	하락
18	7	Course_9	A	4	2	2	2	상승
19	8	Course_10	B	3	[NULL]	1	[NULL]	하락
20	8	Course_11	C	2	3	1	-1	하락
21	9	Course_11	C	2	[NULL]	3	[NULL]	하락
22	9	Course_12	D	1	2	3	-1	하락
23	9	Course_13	A	4	1	3	3	상승

25. orders 테이블의 주문을 order_id 순으로 정렬하여 누적 주문금액과 3개 주문 이동평 균 계산 (ROWS BETWEEN 사용)

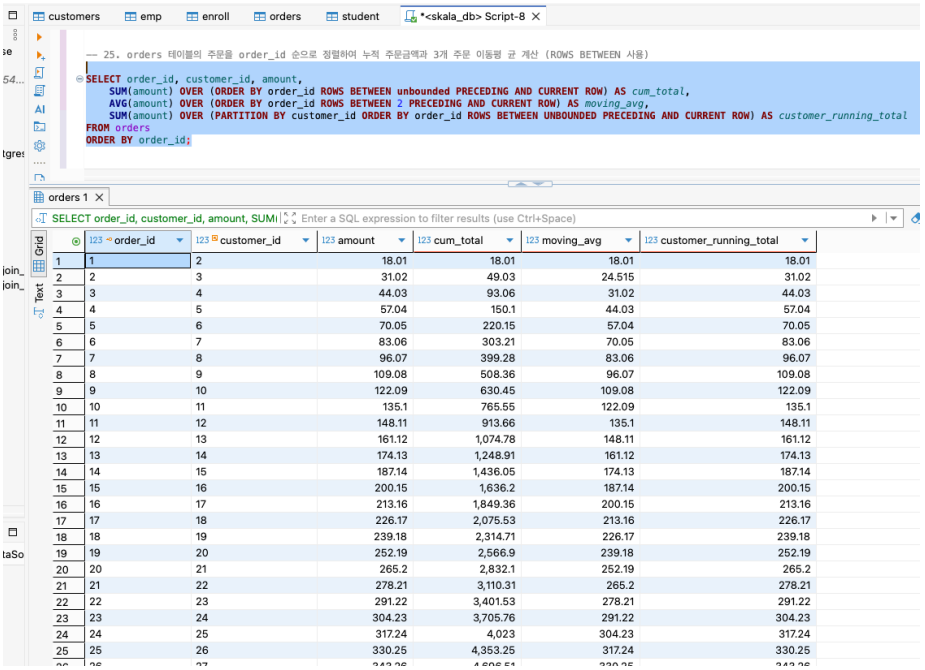
요구조건

- SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW)로 누적합 계산
- AVG(amount) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW)로 3개 이동평균 계산
- customer_id별로 PARTITION을 나눠 고객 별 누적 구매금액도 함께 계산
- 누적합이 전체 합의 50%를 초음하는 첫 번 째 order_id를 찾는 쿼리를 작성

코드

```
SELECT
    order_id,
    customer_id,
    amount,
    SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN unbounded PRECEDING AND CURRENT
ROW) AS cum_total,
    AVG(amount) OVER (ORDER BY order_id ROWS BETWEEN 2 PRECEDING AND CURRENT ROW) AS
moving_avg,
    SUM(amount) OVER (PARTITION BY customer_id ORDER BY order_id ROWS BETWEEN UNBOUND
ED PRECEDING AND CURRENT ROW) AS customer_running_total
FROM orders
ORDER BY order_id;
```

결과



	order_id	customer_id	amount	cum_total	moving_avg	customer_running_total
1	1	2	18.01	18.01	18.01	18.01
2	2	3	31.02	49.03	24.515	31.02
3	3	4	44.03	93.06	31.02	44.03
4	4	5	57.04	150.1	44.03	57.04
5	5	6	70.05	220.15	57.04	70.05
6	6	7	83.06	303.21	70.05	83.06
7	7	8	96.07	399.28	83.06	96.07
8	8	9	109.08	508.36	96.07	109.08
9	9	10	122.09	630.45	109.08	122.09
10	10	11	135.1	765.55	122.09	135.1
11	11	12	148.11	913.66	135.1	148.11
12	12	13	161.12	1,074.78	148.11	161.12
13	13	14	174.13	1,248.91	161.12	174.13
14	14	15	187.14	1,436.05	174.13	187.14
15	15	16	200.15	1,636.2	187.14	200.15
16	16	17	213.16	1,849.36	200.15	213.16
17	17	18	226.17	2,075.53	213.16	226.17
18	18	19	239.18	2,314.71	226.17	239.18
19	19	20	252.19	2,566.9	239.18	252.19
20	20	21	265.2	2,832.1	252.19	265.2
21	21	22	278.21	3,110.31	265.2	278.21
22	22	23	291.22	3,401.53	278.21	291.22
23	23	24	304.23	3,705.76	291.22	304.23
24	24	25	317.24	4,023	304.23	317.24
25	25	26	330.25	4,353.25	317.24	330.25

누적합이 전체 합의 50%를 초음하는 첫 번 째 order_id를 찾는 쿼리를 작성

코드

```
WITH cum AS (
    SELECT
        order_id,
        SUM(amount) OVER (ORDER BY order_id ROWS BETWEEN UNBOUNDED PRECEDING AND CURR
ENT ROW) AS cum_total,
        SUM(amount) OVER ()
AS total_sum
    FROM orders
)
SELECT order_id
FROM   cum
WHERE  cum_total >= total_sum * 0.5
```

```
ORDER BY order_id
LIMIT 1;
```

결과

